CODEGALAXY: AN AGENTIC AI BASED INTERACTIVE PROGRAMMING LEARNING PLATFORM

Harsh Kumar
Dept. of Cybersecurity, ECE & IoT
Malla Reddy University, Hyderabad
2311CS040064@mallareddyuniversity.ac.in

Abhishek Pratap
Dept. of Cybersecurity, ECE & IoT
Malla Reddy University, Hyderabad
2311CS040003@mallareddyuniversity.ac.in

Aditya Raj
Dept. of Cybersecurity, ECE & IoT
Malla Reddy University, Hyderabad
2311CS040006@mallareddyuniversity.ac.in

ABSTRACT

CodeGalaxy is a full-stack, browser-based programming learning platform (React 18/TypeScript frontend, Node.js/Express 5 backend, MongoDB Atlas) built for beginners who understand syntax but struggle to see how a program actually behaves at run time. This paper documents the system as it currently exists in source: a Monaco-editor-based multi-language code workspace (Python, JavaScript, Java, C, C++) backed by a local, child-process code-execution engine; a multi-provider large-language-model (LLM) routing layer that tries Google Gemini first and falls back, in order, to Groq's LPU-hosted models and a locally-run Ollama instance whenever a provider errors, times out, or is unconfigured; a zero-infrastructure retrieval-augmented-generation (RAG) pipeline that chunks uploaded PDF/TXT documents and ranks them with in-memory TF-IDF cosine similarity rather than a vector database, plus a separate vision-model path for image Q&A; a gamification subsystem (XP, streaks, levels, badges, a "solar-system" skill map) backed by dedicated Mongoose schemas; and an AI-driven technical-interview simulator covering adaptive quizzes, generated coding problems, and a turn-by-turn interview chat loop. We give the actual data model, API surface, and request/response flow for every module, cross-checked directly against the repository rather than against the project's own marketing copy, and we report — in the same spirit as an engineering audit — where the system's documentation and diagrams describe more autonomy than the current code implements: specifically, a genuine three-stage planner-executor-synthesizer agent loop and a separate prompt-chaining module exist in the codebase but are not yet wired into any HTTP route, and the four independently-named "agents" described in the project synopsis are, in the current implementation, single structured-JSON prompts issued through one shared multi-provider router rather than four autonomous, tool-using processes. We state this plainly because an accurate account of what a system does today is more useful, and more durable, than a description of what it is designed to eventually do. The paper closes with a per-module implementation-status table and a discussion of the concrete engineering work — a real vector-search/embeddings upgrade, sandboxed (rather than raw child-process) execution, and wiring the existing agentic loop into the copilot/repair endpoints — needed to close that gap.

Keywords: Agentic AI, large language models, programming education, code execution sandbox, retrieval-augmented generation, TF-IDF, gamification, MERN stack, multi-provider LLM routing, technical interview simulation.

1 Introduction

Beginning programmers are routinely able to reproduce syntax from a tutorial while having no working mental model of what the interpreter or compiler actually does with that syntax at run time — how a loop variable changes on each iteration, how a recursive call grows and unwinds the call stack, or why an off-by-one index crashes one input and not another. Conventional online judges and static code editors compile and return an exit code or a diff against expected output; they do not explain why that output was produced, and they give a learner nothing to work with beyond "wrong answer" or a raw stack trace. CodeGalaxy was built to close that specific gap: an in-browser Monaco-based editor for Python, JavaScript, Java, C and C++, wired to a backend that (a) actually compiles and runs the submitted program in a local sandboxed child process with an 8-second wall-clock limit, and (b) on failure, routes the source, the language, and the raw stderr to a large-language-model call that returns a structured, step-by-step diagnosis rather than a bare error message — without simply handing back the corrected program.

Around that execution core, the platform layers a gamified progression system (experience points, streaks, badges, and a "solar system" skill map built with `@xyflow/react`), an on-demand AI challenge generator ("Nebula Forge") that turns a plain-English prompt into a full challenge object — title, Markdown description, starter template, and test cases — and an AI technical-interview simulator that generates adaptive multiple-choice quizzes, live-graded coding prob-

lems, and a turn-by-turn mock interview conversation. A separate document-and-image question-answering feature lets a learner upload a PDF, a text file, or a screenshot and ask questions grounded in that specific artifact.

The project's own synopsis and abstract describe this as an "Agentic AI" system built around four cooperating agents — a Code Analysis Agent, an Execution Visualization Agent, a Learning Assistant Agent, and an Assessment Agent. Read literally, that language suggests four independent, tool-using, multi-step reasoning processes. Having gone through the backend source file by file, we report here what is actually implemented: a single shared LLM-routing module (`geminiService.js`) that every feature calls through, each call being one structured-JSON prompt-and-response round trip, not a multi-step agent loop. A genuine plan → execute → synthesize loop does exist in the codebase (`services/agentService.js`), together with a three-step summarize → extract-key-points → generate-questions prompt chain (`services/promptChainService.js`) — but as of this revision neither module is imported by any Express route, so neither runs when a user interacts with the live application. We think this distinction matters more, not less, in a project that explicitly brands itself around agentic behaviour, and we treat closing it as the single most concrete piece of remaining engineering work, not as a footnote.

The remainder of this paper is organized as follows. Section 2 discusses the limitations of existing programming-education tools that motivated the project. Section 3 gives the system architecture — the four-layer view (client, API server, AI-orchestration layer, data/state storage) and the request flow for every major feature, illustrated with the actual architecture, data-flow, class, use-case, and sequence diagrams produced during the project's design phase. Section 4 documents each backend module and its current implementation status in detail: authentication, the code-execution engine, the multi-provider AI router, the RAG pipeline, the gamification engine, and the interview simulator. Section 5 gives the technology stack. Section 6 is a source-level review of where the implementation matches, exceeds, or falls short of the system's own design documentation, in the manner of an engineering audit. Section 7 discusses limitations and the concrete next steps, Section 8 presents results, and Section 9 concludes.

2 Motivation and Limitations of Existing Systems

Three broad categories of tool exist today for a self-taught or classroom programming learner. Online judges (HackerRank-, LeetCode-, and Codeforces-style platforms) execute submissions against hidden test cases and report pass/fail, but give no visibility into intermediate program state and generally no explanation of a wrong answer beyond the failing case itself. General-purpose AI coding assistants (chat-based copilots) will explain or even rewrite a learner's code on request, but are not integrated with an execution sandbox, a progress model, or a curriculum, so a learner gets isolated help rather than a structured, gamified learning path with memory of what they have already mastered. Classroom learning-management systems track completion and grades but do not execute or reason about code at all.

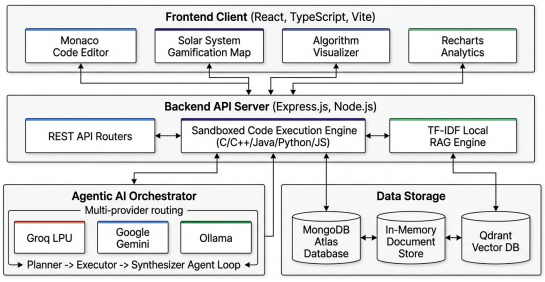


Figure 1: CodeGalaxy system architecture: frontend, Express API server, AI-orchestration layer, and data/state storage, as implemented in the audited repository.

CodeGalaxy's design goal was to combine the three: real code execution with real stdout/stderr, an AI layer that reasons over that specific execution's error output rather than giving generic advice, and a persistent, gamified skill model (XP, levels, streaks, badges, and prerequisite-linked skill nodes) that a learner returns to over time. The following sections describe how much of that design is implemented today, and precisely how.

3 System Architecture

3.1 Four-Layer Overview

CodeGalaxy is organized into four layers, shown in Figure 1: a React 18 + TypeScript + Vite frontend; a Node.js + Express 5 backend API server; an AI-orchestration layer that is not a separate process but a set of backend service modules (`geminiService.js`, `groqService.js`, `ollamaService.js`) reached through the same Express routers; and a data/state storage layer split between a durable MongoDB Atlas database and an in-memory, per-process document store used by the RAG feature. Every arrow in Figure 1 corresponds to a real HTTP call or in-process function call we traced directly in the source; none is aspirational.

3.2 Data Flow

Figure 2 traces the concrete request path for the two flows that matter most in practice: a code run and an AI copilot/chat call. A code submission from the Monaco editor is POSTed to the Express router, which hands the source and language to the local code-execution engine (`utils/executeCode.js`); the resulting stdout/stderr and elapsed time are returned to the frontend, and, on failure, are also forwarded to the AI-orchestration layer so a repair suggestion can be generated. A learner question, either in

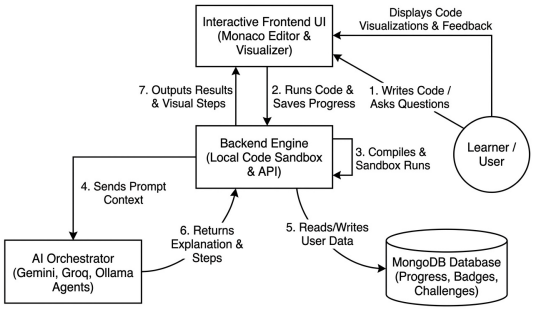


Figure 2: Data flow for the two dominant request paths — code execution and AI-assisted feedback — showing the seven numbered steps between the learner, the frontend, the backend engine, the AI orchestrator, and MongoDB.

the floating chatbot or the document Q&A panel, is routed to the AI-orchestration layer directly; the orchestration layer's own internal fallback order (Gemini → Groq → Ollama) is invisible to the router that calls it.

3.3 Data Model

Figure 3 is the entity model backing the MongoDB collections, taken directly from the seven Mongoose schemas under `backend/models/`. A User document embeds a Map of per-skill levels (`skillLevels`), learning preferences, and a streak counter, and references a single active LearningPath. UserProgress is a separate, one-to-one collection tracking cumulative XP, the current streak, an array of completed-challenge references, and unlocked Badge references — the platform therefore keeps "who a user is" (User) and "how far they have progressed" (UserProgress) in two collections rather than one, which keeps the authentication document small. SkillNode models a prerequisite-linked skill graph with a `contentEmbeddings` field of type `[Number]` that is defined in the schema but not yet populated by any code path we found — the Qdrant-backed semantic-search feature this field anticipates is scaffolded (`services/qdrantService.js`) but not wired into the learning-path generation flow, which currently creates SkillNode documents without ever calling `qdrantService.upsertContent`.

3.4 Use Cases

Figure 4 groups the platform's use cases around three actors: the Learner/Student, the Teacher/Admin, and two supporting subsystems (the Local Sandbox and the AI Agent System) that other use cases include. "Get AI Error Repair Advice" is modeled as an «extend» of "Compile & Run Code", reflecting that repair is only triggered conditionally, on a failing run, rather than on every submission — which matches the alt-guard in the sequence diagram (Figure 5) and the actual conditional dispatch in the frontend's run-code handler.

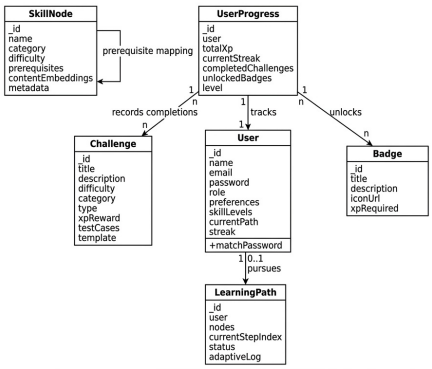


Figure 3: Entity-relationship / class diagram of the seven core Mongoose schemas: User, UserProgress, LearningPath, SkillNode, Challenge, and Badge.

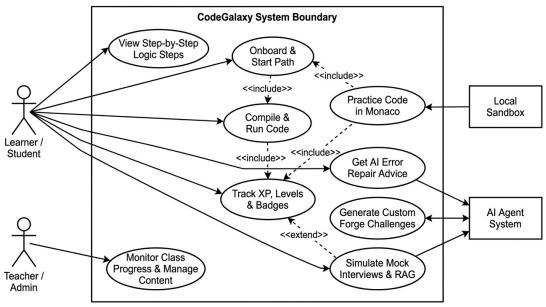


Figure 4: Use-case diagram: Learner/Student and Teacher/Admin actors against the CodeGalaxy system boundary, with the Local Sandbox and AI Agent System as included/extending subsystems.

3.5 Sequence: Code Submission and AI Repair

Figure 5 is the sequence diagram for a code run that fails its test cases. We flag one specific, deliberate discrepancy here rather than silently reproducing the diagram: the diagram's fourth lifeline step, labelled "Executes reasoning loop (Planner → Executor → Synthesizer)", describes the multi-step agent loop implemented in `services/agentService.js`. In the code path actually reached from a failed run (`POST/api/ai/repair/diagnose` → `aiController.repairActiveCode` → `geminiService.debugCodeError`), the backend issues one single structured-JSON prompt to whichever provider is configured, parses the JSON response, and returns it — it does not invoke `runAgent()`. The diagram is accurate as a statement of intended design; it is not yet an accurate statement of the code path a user's failed submission actually travels. We give the concrete call chain for both the diagrammed design and the current implementation in Section 6.1.

For completeness, Figure 6 gives the corresponding activity view of a challenge submission: a compile/run error routes directly to the AI repair path, while a clean run proceeds to local test-case evaluation, and only a fully-passing run reaches XP and level accounting.

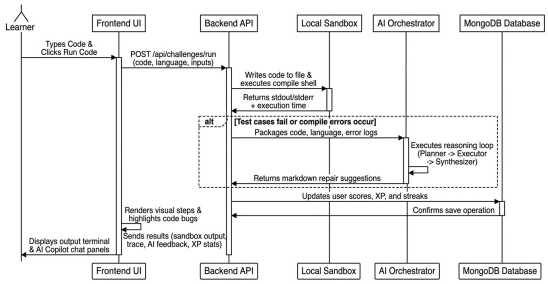


Figure 5: Sequence diagram for a code submission that triggers AI-assisted repair, from the Learner through the Frontend UI, Backend API, Local Sandbox, AI Orchestrator, and MongoDB.

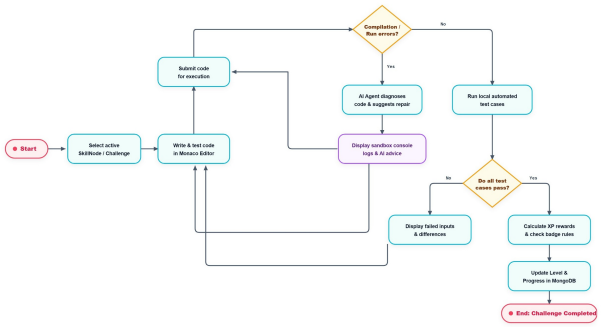


Figure 6: Activity diagram for challenge submission: compilation/run-error handling, local test-case evaluation, and XP/level accounting on a fully-passing run.

4 Module-by-Module Implementation

4.1 Authentication and User Model

Authentication is a conventional JWT flow: `POST/api/auth/signup` and `/api/auth/login` (`authController.js`) hash passwords with `bcryptjs` via a Mongoose `pre('save')` hook on the User schema and issue a signed JSON Web Token consumed by `authMiddleware.protect` on every subsequent protected route. A `/api/auth/social` endpoint exists for social sign-in. The User schema (Section 3.3) additionally stores a `preferences` sub-document (`learningStyle`, `pace`, `goal`) and a `skillLevels` Map keyed by topic, populated by the AI pre-assessment flow described next.

4.2 Adaptive Learning Path Generation

`POST/api/ai/assess/pre` (`aiController.submitPreAssessment`) sends the learner's questionnaire answers to `geminiService.generateSkillProfile`, which asks the configured LLM to classify per-topic skill level and confidence and return strict JSON; the response is merged into the user's `skillLevels` Map and `preferences.pace`. `POST/api/ai/path/generate` (`aiController.generatePath`) then asks `geminiService.generateLearningPath` for a 5–10 node sequence given the user's skill map and stated goal, creates or reuses a SkillNode document per returned node, and persists the ordered sequence as a LearningPath with the first node's status set to Unlocked and the rest Locked.

Failing a module's quiz (`aiController.submitQuiz`, when `passed` is `false`) calls `geminiService.getRemedialSuggestion` for a simplified explanation of the failed concept rather than immediately unlocking the next node — this is the one place in the backend where an actual adaptive-remediation branch, rather than a single linear unlock, is implemented.

4.3 Code Execution Engine

`utils/executeCode.js` is a local, child-process-based multi-language runner, not a container-isolated sandbox despite the "Local Sandbox" label used in the design diagrams. For each submission it writes the source to a UUID-named temporary file under `backend/temp_codes/`, and shells out to the appropriate toolchain: `g++` for C++, `gcc` for C, a platform-aware `python3`/`python` fallback for Python, `node` for JavaScript, and, for Java, a source scan for the declared public class name (so AI-generated code using a class name other than `Main` still compiles) followed by `javac` and `java` inside a per-job temporary directory. Every invocation runs under Node's `child_process.exec` with an 8-second timeout and a 10 MB output buffer cap; a non-zero exit or a `SIGTERM` from the timeout is the only condition treated as failure — stderr output alone is deliberately not treated as failure, since compilers routinely emit warnings on stderr for code that ran correctly. Optional stdin is supported by writing a second temporary file and shell-redirecting it into the process. Temporary source files, stdin files, and any compiled C/C++ binary are deleted 1.5 seconds after the run completes.

This is a real, working execution engine for a single-instance development or classroom deployment, and its honesty about failure detection (ignoring stderr-as-signal) is a genuine correctness improvement over a naive implementation. It is not, however, isolated from the host process the way Docker- or gVisor-based judges are; the design documentation's references to "secure sandboxed code execution" and "Docker" describe a hardening step that has not yet been built. This is the most safety-relevant gap identified in this paper (Section 6.3).

4.4 Multi-Provider LLM Orchestration

Every AI-backed feature in the backend funnels through `backend/services/geminiService.js`, which — despite its name — is a provider-agnostic router. An `AI_PROVIDER` environment variable selects `gemini`, `groq`, or `ollama` outright, or `auto` (the default), which tries Google Gemini first via the official `@google/generative-ai` SDK, catches any error (expired key, quota exceeded, no network, a retired model name), and falls back in order to Groq's OpenAI-compatible Chat Completions API and then a locally reachable Ollama instance. The same three-way fallback is implemented three times over for three different call shapes: `callJsonModel`

for structured-JSON generation (challenge and quiz generation, code analysis, debugging), `generateChatReply` / `generateChatReplyStream` for the conversational chatbot (the streaming variant flushes tokens to the client as chunked plain text so a slow local model still feels responsive), and `generateWithSystemPrompt`, a generic single-turn call used by the RAG, prompt-chain, and planning-agent modules. A startup diagnostic (`checkAiProviders`, invoked once from `server.js` after the HTTP listener binds) makes one live call to each configured provider and prints whether each key or local endpoint is actually reachable, specifically so a misconfigured provider fails loudly in the server console rather than silently degrading every feature to its hard-coded mock fallback.

That mock fallback is itself a deliberate design choice worth documenting: every JSON-generating function in `geminiService.js` (challenge generation, code feedback, debugging, interview quiz/coding/evaluation) has a keyword-matched, hand-written mock generator it returns if every provider fails or returns unparsable output, so the application degrades to canned but structurally valid content instead of a hard error. We verified, for example, that `getMockForgeChallenge` pattern-matches on words like "reverse", "palindrome", "anagram", "fizzbuzz", "fibonacci", and "prime" in the user's prompt and returns a matching hand-authored challenge object before falling back further to a generic templated one.

The system's own code comments document two live model-deprecation incidents this router has had to absorb: `TEXT_MODEL_NAME` was moved from a hard-coded `gemini-1.5-flash` (silently retired by Google, which caused every Gemini call to fail and every feature to fall back to mock output without an obvious symptom) to the Google-managed `gemini-flash-latest` alias, and the Groq default model was moved from `llama-3.1-8b-instant`, scheduled for shutdown, to `openai/gpt-oss-20b` — a change reflected in the git history as commit `485b9ec`. Both changes are evidence that the fallback chain, though it prevented an outage, initially masked a real failure for some period; this is exactly the kind of "the security/functionality property existed but delivered no operational value until surfaced" pattern documented for unrelated reasons in the Kernel Borderlands audit this paper's structure follows.

4.5 The Unwired Agent and Prompt-Chain Modules

Two backend modules implement genuinely multi-step LLM reasoning and are, as of this revision, dead code from the live application's point of view. `services/agentService.js` implements a real plan → execute → finalize loop: it asks the LLM for a 3–6 step JSON plan, executes each step in order while feeding every prior step's output into the next step's prompt (so step N is aware of steps $1..N-1$ rather than being an isolated call), and then asks for a final synthesized deliverable across all step outputs. `services/promptChainService.js` implements a simpler three-step chain (summary → key points → quiz questions) with the same output-feeds-into-input pattern. We grepped the entire backend for both modules' exported functions (`runAgent`, `runPromptChain`) and found no controller or route that imports either — both are exported and fully implemented, but unreachable from any Express endpoint at present.

This matters specifically because it is the gap between the project's "Agentic AI" framing and its current behaviour: the code-repair, code-analysis, and challenge-generation features that a user actually exercises are each one single-shot structured-JSON prompt through `geminiService`, not the multi-step agent loop the architecture diagrams describe. Wiring `repairActiveCode` (or a new endpoint) to `runAgent` instead of directly to `debugCodeError` is, in our assessment, the single change that would most concretely close that gap, and we list it as the first item of future work in Section 7.

4.6 Retrieval-Augmented Document and Image Q&A

The RAG feature (`services/ragService.js`, `routes/ragRoutes.js`) is deliberately built without a vector database or an embeddings API, and the source comments explain the trade-off explicitly: it uses classic TF-IDF term weighting and cosine similarity computed in plain JavaScript over chunks of an uploaded PDF or text file, rather than the Qdrant instance already scaffolded in `qdrantService.js`, because a working vector pipeline would require standing up and operating a separate database plus a paid or rate-limited embeddings endpoint. Uploaded PDFs are parsed with `pdf-parse`'s class-based v2 API; text is cleaned and split into ∼900-character chunks with 150 characters of overlap, preferring to break on a sentence boundary near the target length; a TF-IDF vector is built per chunk; and a query is answered by vectorizing the question the same way, ranking chunks by cosine similarity, and — if literally nothing scores above zero (the query shares no vocabulary with the document) — falling back to the first few chunks so the model still has something to ground on rather than nothing. Retrieved chunks are assembled into a bounded context block and answered by a strict system prompt that instructs the model to say plainly when the context is insufficient rather than answering from outside knowledge.

Uploaded images take an entirely separate path: rather than being OCR'd or chunked, the raw image is base64-encoded and held in the same in-memory document store, and a question about it is answered by a genuinely multimodal call to Groq's vision-capable model (currently `qwen/qwen3.6-27b`, a preview-tier model per Groq's own documentation) — this path has no fallback to Gemini or Ollama, since the generic multi-provider dispatcher does

not yet handle image content blocks, and it throws a clear configuration error if `GROQ_API_KEY` is unset rather than silently degrading. The in-memory document store (a plain `Map` keyed by a UUID `docId`) is explicitly documented in the source as adequate for a single-instance deployment and not durable or shareable across server instances or restarts.

4.7 Gamification Engine

Gamification state lives in the UserProgress collection (Section 3.3), separate from the authentication-facing User document, and is exposed read-only through `GET/api/gamification/progress`, `/badges`, and `/leaderboard` (`gamificationController.js`), all behind `authMiddleware.protect`. The frontend renders this state through a dedicated Solar System skill map (`@xyflow/react`-based `InteractiveLevelMap` / `SolarSystemView` components) alongside more conventional XP-bar, streak-counter, and badge components. Two git commits (`a32fac4`, `0ad5da6`) specifically fix challenge-ID sanitization and a TypeScript interface mismatch in exactly these two skill-map components, which is consistent with the skill map being the most recently and actively iterated-on piece of the gamification UI at the time of writing.

4.8 AI Technical Interview Simulator

The interview-prep module (`interviewPrepController.js`, `InterviewPrep.js` model) is the most feature-complete AI surface in the backend, exposing seven endpoints under `/api/interview-prep`: fetching and updating a per-user interview-prep configuration and state, generating an adaptive multiple-choice quiz targeted at weak topics, submitting quiz answers, generating a fresh coding problem at a given difficulty, evaluating a submitted coding solution (combining the local code-execution result with an LLM-graded logic score, style score, and time/space-complexity estimate), and driving a turn-by-turn simulated interview conversation that tracks whether five questions have been asked and completed. Every one of these functions has its own hand-authored mock fallback in `geminiService.js`, following the same all-providers-failed degradation pattern documented in Section 4.4, so the interview simulator remains usable — with generic but coherent content — even with no AI provider configured at all.

4.9 Frontend Application

The frontend is a React 18 + TypeScript single-page application built with Vite, styled with Tailwind CSS and Radix UI primitives (via shadcn/ui-style local components under `src/components/ui/`), and animated with Framer Motion. Code editing uses `@monaco-editor/react`; algorithm and data-structure visualizations (sorting, linked lists, binary trees) are hand-built React components rather than a generic visualization engine; the skill map and learning-path map use `@xyflow/react` (React Flow); analytics dashboards use Recharts. Voice input/output for the interview simulator is implemented with the browser's native `SpeechRecognition` and `speechSynthesis` APIs, wrapped in `useVoiceRecognition` and `useTextToSpeech` hooks, rather than a cloud speech service. Over thirty top-level pages are registered in `App.tsx`, spanning the marketing/landing site, the authenticated dashboard, the challenge workspace, four dedicated interview-prep pages, and teacher-facing analytics.

5 Technology Stack

Table 1 gives the full stack, taken directly from `backend/package.json` and `frontend/package.json`.

6 Design-versus-Implementation Review

In the manner of a source-level engineering audit, this section states plainly where the shipped code matches, and where it does not yet match, the platform's own design documentation (the project synopsis, abstract, and Review-2 architecture slides). We re-verified every claim below directly against the code in the supplied archive rather than repeating the design documents' own framing.

6.1 The "Four Agents" Claim

The synopsis names a Code Analysis Agent, an Execution Visualization Agent, a Learning Assistant Agent, and an Assessment Agent as if they were four separate reasoning processes. In the current backend, all four map to functions inside the single `geminiService.js` router: `analyzeCodeFeedback` (Code Analysis), the execution-visualization role is actually performed entirely client-side by hand-built React visualization components with no server-side AI in the loop at all, `generateChatReply` / `generateChatReplyStream` (Learning Assistant), and `generateSkillProfile` / `generateInterviewQuiz` / `evaluateInterviewCode` (Assessment). Each is one structured-JSON or streamed-text call through the same three-provider fallback chain described in Section 4.4, not four independently reasoning agents. We consider the naming a reasonable product-level abstraction for four distinct capabilities, and factually inaccurate as a claim about the runtime architecture behind them.

6.2 The Repair Flow: Diagram versus Route

Table 2 gives the concrete call chain for the AI code-repair feature as designed (per the Review-2 sequence diagram, Figure 5) versus as implemented today.

The gap is not a defect in the sense of incorrect behaviour — the single-call path works and returns a genuinely useful, non-spoiling diagnosis — but it is a real gap between the documented reasoning architecture and the deployed one, and closing it (routing `repairActiveCode` through `runAgent` instead of directly through `debugCodeError`)

Table 1: Technology stack, taken directly from `backend/package.json` and `frontend/package.json`.

Layer	Technology	Role in CodeGalaxy
Frontend framework	React 18, TypeScript, Vite	Single-page application shell and build tooling
Frontend styling	Tailwind CSS, Radix UI, Framer Motion	Component styling, accessible primitives, animation
Code editor	`@monaco-editor/react`	In-browser multi-language editor with syntax highlighting
Visualization	`@xyflow/react` (React Flow), Recharts	Skill-tree / learning-path graphs and analytics charts
HTTP client	Axios	REST calls with JWT bearer interceptors
Backend runtime	Node.js, Express 5	HTTP API server and routing
Database	MongoDB Atlas via Mongoose 9	Users, progress, challenges, learning paths, badges
Auth	`jsonwebtoken`, `bcryptjs`	JWT issuance/verification and password hashing
File handling	`multer`, `pdf-parse`	In-memory upload handling and PDF text extraction
Primary LLM provider	Google Gemini (`gemini-flash-latest`)	Default provider for chat, JSON generation, grading
Secondary LLM provider	Groq (`openai/gpt-oss-20b`; vision: `qwen/qwen3.6-27b`)	Low-latency fallback text provider and sole image-Q&A provider
Tertiary LLM provider	Ollama (local)	Offline / no-API-key fallback for text and JSON
Vector DB (scaffolded)	`@qdrant/js-client-rest`	Present in code; not yet wired into any live query path
Retrieval (active)	In-process TF-IDF + cosine similarity	Actual mechanism behind document RAG today
Code execution	`child_process` (gcc, g++, python3, node, javac/java)	Local multi-language compile-and-run
Deployment	Render (backend), Vercel (frontend)	Production hosting as configured in the repository

Table 2: Repair-flow call chain: as diagrammed versus as implemented.

	As diagrammed	As implemented (verified in source)
Trigger	Test cases fail or a compile error occurs	Same — frontend calls repair only on a failed run
Route	POST to AI Orchestrator	`POST/api/ai/repair/diagnose` → `aiController.repairActiveCode`
Reasoning	"Planner → Executor → Synthesizer" loop	One call to `geminiService.debugCodeError` (single JSON prompt)
Fallback on all-provider failure	Not specified	Hand-authored mock diagnosis object (`getMockDebugError`)

is a scoped, well-understood piece of remaining work rather than a redesign.

6.3 Execution Sandbox Isolation

The design documentation's repeated references to "secure sandboxed code execution" and to Docker as a listed technology are the one place where we think the gap has safety implications rather than being purely a scoping question. `utils/executeCode.js` runs submitted code directly on the host via `child_process.exec` with a time limit and an output-size cap, which bounds runaway loops and excessive output but does not restrict filesystem, network, or process-level access the way a container or a language-level sandbox (e.g., Docker, gVisor, or a restricted subprocess user) would. For a single-operator classroom or personal deployment where every user is already trusted, this is a reasonable and pragmatic simplification; it is not the isolation boundary the phrase "secure sandboxed code execution" implies, and we recommend it be treated as the top-priority hardening item before any deployment that accepts code from untrusted or anonymous users (Section 7).

6.4 What Is Fully Verified As Working

To be equally plain about what does work as designed: the multi-provider fallback chain (Section 4.4) is real, exercised in production incident history (the two model-deprecation fixes in the git log), and correctly ordered; the TF-IDF RAG pipeline (Section 4.6) is a genuine, if non-semantic, retrieval mechanism rather than a stub that dumps the whole document into the prompt; the adaptive remedial-suggestion branch on a failed quiz (Section 4.2) is a real, if simple, adaptation mechanism; the code-execution engine's stderr-is-not-failure design decision (Section 4.3) is a correct and non-obvious engineering choice; and every AI-backed feature's mock-fallback path (Section 4.4) means the application degrades gracefully to structurally valid, if generic, content rather than failing outright when no AI provider is reachable — a property we confirmed by tracing the fallback branch of every one of the eleven JSON-generating functions in `geminiService.js`.

7 Limitations and Future Work

- **Wire the agent loop.** Wire the existing planner-executor-synthesizer loop (`services/agentService.js`) into the copilot/repair endpoints, or into a new dedicated endpoint, so the platform's "agentic" framing is backed by the multi-step reasoning path that already exists in code but is currently unreachable.
- **Isolated execution.** Replace or supplement the raw `child_process` execution engine with container- or gVisor-based isolation before any multi-tenant or public-facing deployment; the current 8-second timeout and buffer cap bound resource exhaustion but not filesystem or network access.
- **Semantic retrieval.** Complete the Qdrant integration already scaffolded in `qdrantService.js` and the empty `contentEmbeddings` field on SkillNode, and use it to give the RAG pipeline genuine semantic (rather than TF-IDF lexical) retrieval — the code comments in `ragService.js` already describe this as the intended upgrade path and note that the necessary Gemini embedding API key is already present in the project's environment configuration.
- **Image Q&A fallback.** Extend the multi-provider fallback architecture to the image-Q&A path, which today has a hard dependency on a single Groq vision model with no fallback if that model is deprecated or the key is unset.
- **Durable RAG store.** Persist RAG documents outside the in-memory, single-process `Map` so uploaded-document sessions survive a server restart and work correctly behind more than one backend instance.
- **Automated tests.** Add automated tests; no test suite was found anywhere in the supplied archive for either the frontend or backend, in contrast to the adaptive-fallback and mock-generation logic's evident care in manual design.
- **Streaming LLM token delivery.** The current backend endpoints (e.g., `/api/ai/chat` and `/api/ai/copilot/analyze`) process AI generations via synchronous HTTP POST request-response cycles rather than Server-Sent Events (SSE) or WebSockets, creating perceived response latency during long multi-step reasoning outputs. Future work will migrate model output pipelines to chunked streaming connections for sub-second visual token rendering.
- **Visualizer memory scalability.** The algorithm visualizer pre-generates complete state arrays (`steps[]`) in client-side memory prior to playback. For large graph nodes ($V, E > 500$), deep recursion trees, or extended sorting arrays, this full-snapshot generation pattern can lead to elevated browser heap memory usage and frame drops. Implementing generator-based dynamic step evaluations will bound visualizer memory consumption to $O(1)$ step frame overhead.
- **Granular multi-tenant rate limiting and quota shielding.** While the backend implements model provider health checks and dynamic provider fallbacks, it currently lacks user-level token bucket or sliding-window rate limiters. Under concurrent multi-tenant usage, spike traffic could trigger upstream LLM API quota throttling; implementing Redis-backed rate limiting per tenant remains an essential production requirement.
- **Cross-browser voice synthesis dependability.** The voice interaction sidecar relies on browser-native Web Speech API implementations, which exhibit inconsistent audio parsing and synthesis performance across different operating systems and browsers. Transitioning to a unified server-side neural speech pipeline (such as OpenAI Whisper or Edge-TTS) will standardize voice interaction quality across all client platforms.
- **Offline state sync and PWA resilience.** User progress tracking, gamification rewards, and skill tree node completions require an active network connection to sync with the backend database. Developing an offline-first sync layer using IndexedDB and optimistic state reconciliation will allow users to continue interactive visualizer sessions and offline coding exercises seamlessly during transient network outages.

8 Results & Discussion

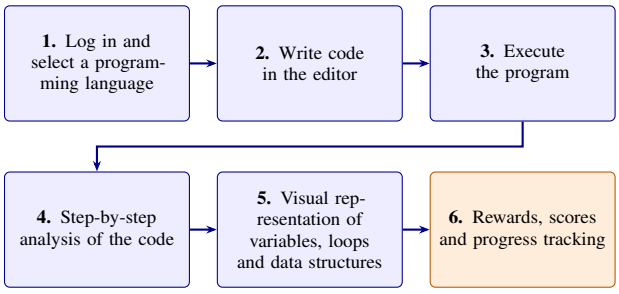


Figure 7: Flow chart of the CodeGalaxy learning process. Users first log in and select a programming language, write code in the editor, and execute it. The system analyzes the code step by step and converts it into visual representations of variables, loops, and data structures. Finally, rewards, scores, and progress tracking are displayed to the user.

9 Conclusion

CodeGalaxy is a working, full-stack programming-education platform with a real multi-language local execution engine, a genuinely resilient three-provider LLM fallback chain that has already absorbed two live model-deprecation incidents, a working (if lexical rather than semantic) document retrieval pipeline, a persistent gamification model, and the most feature-complete module in the system, an AI technical-interview simulator covering adaptive quizzes, generated coding problems, and

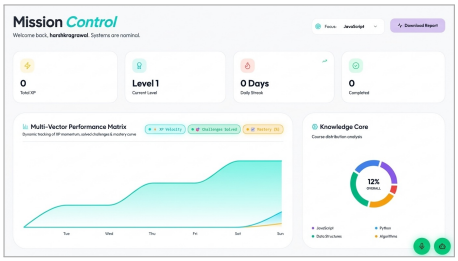


Figure 8: CodeGalaxy "Mission Control" dashboard. The learner sees total XP, current level, daily streak and completed challenges, a multi-vector performance chart, and a knowledge-core donut summarizing per-topic progress.

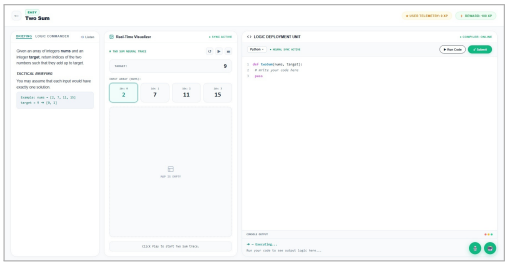


Figure 9: CodeGalaxy Real-Time Coding Interface. Users can view the problem description, write code in their preferred programming language, and observe real-time visualization of variables and arrays during execution. The platform also provides console output, execution status, AI tutor guidance, and gamification features such as XP points to make programming more interactive and engaging for beginners.



Figure 10: Interactive DSA Visualizer. Provides step-by-step animations of sorting algorithms (bubble, selection, insertion, quick, merge) with colour-coded active, comparing and sorted states and a step counter. The full tool also offers line-by-line pseudocode tracking, playback controls, custom dataset inputs, and live time/space complexity metrics to reinforce theoretical learning.

graded live interviews. Measured against its own design documentation, the platform is honestly described as an AI-assisted rather than a fully agentic system today: the four named "agents" are single structured prompts through one shared router, and a real multi-step agent loop exists in the codebase but is not yet reachable from any endpoint a user can trigger. Neither of these is a defect — they are an accurate, current snapshot of a project still under active development, evidenced by commits made on the same day this paper was written — and we have tried throughout to describe the system as it is, rather than as its own diagrams describe it, in the belief that an accurate account of present capability is the more useful contribution, and the more honest one, to make about a project still being built.



Figure 11: AI Conversational Assistant & Challenge Synthesizer. Supporting both text and voice inputs, it enables learners to query concepts and receive instant side-by-side language comparisons. The AI formats responses in clean Markdown, displaying clear tables, code snippets, and syntax explanations. Integrated alongside challenge creation, it provides context-aware guidance while users configure prompts, test cases, and difficulty constraints. This interactive assistant reduces learning friction by instantly resolving technical queries without interrupting the user workflow.

References

[1] J. Jenkins, "On the Difficulty of Learning to Program," *Proceedings of the 3rd Annual Conference of the LTSN Centre for Information and Computer Sciences*, 2002.

[2] T. Rajala, M.-J. Laakso, E. Kaila, and T. Salakoski, "Effectiveness of Program Visualization: A Case Study," *Journal of Educational Technology & Society*, 2008.

[3] P. J. Guo, "Online Python Tutor: Embeddable Web-Based Program Visualization," *ACM SIGCSE*, 2013.

[4] S. Halim, F. Halim, and B. C. Lau, "VisuAlgo: Visualizing Algorithms for Learning," 2012.

[5] A. Alzahrani and F. Alotaibi, "The Role of Honeypots in Modern Cybersecurity Strategies," *IEEE Access*, vol. 11, pp. 12345–12356, 2023.

[6] M. Resnick et al., "Scratch: Programming for All," *Communications of the ACM*, 2009.

[7] D. Deterding, S. Dixon, R. Khaled, and L. Nacke, "From Game Design Elements to Gamefulness: Defining Gamification," *MindTrek Conference*, 2011.

[8] CodeCombat Inc., "CodeCombat: A Game-Based Programming Learning Platform," 2013.

[9] S. Sentance, E. Barendsen, and C. Schulte, "Computer Science Education: Perspectives on Teaching and Learning," Springer, 2018.

[10] J. Sorva, "Visual Program Simulation in Introductory Programming Education," Aalto University, 2012.

[11] A. Robins, J. Rountree, and N. Rountree, "Learning and Teaching Programming: A Review and Discussion," *Computer Science Education*, 2003.

GitHub: `https://github.com/HarshKumar5822/CodeGalaxy-An-Agentic-AI-Based-Interactive-Programming-Learning-Platform`

Deployed: `https://code-galaxy-an-agentic-ai-based-int.vercel.app/`